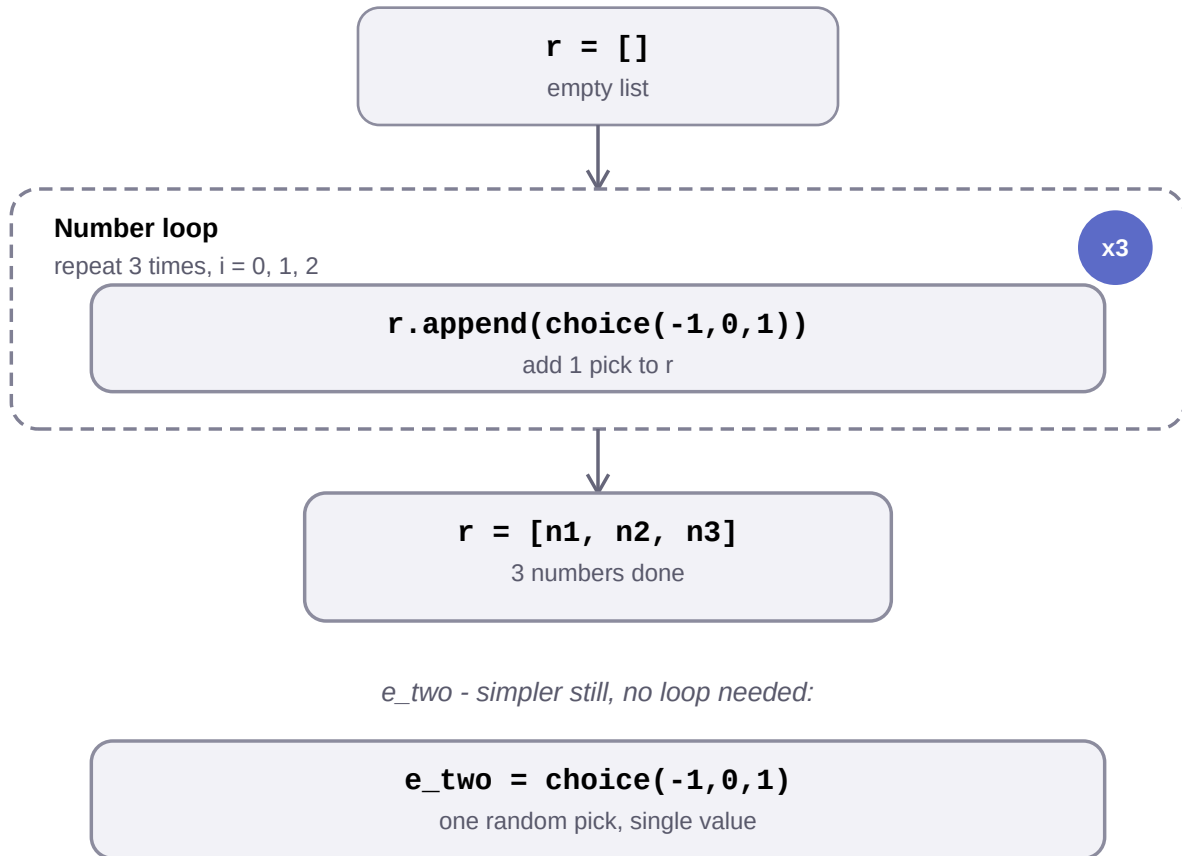


Toy Integer Kyber (3x3)

Code walkthrough: Alice's encapsulation

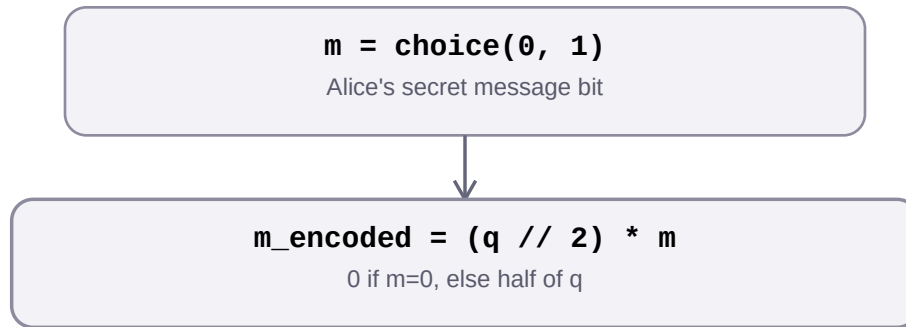
This document accompanies `toy_integer_kyber_3x3.py`.
Alice receives A and b and creates u and v with her new
values of e_one , e_two , r and, most importantly, her
message m , which she encodes and sends alongside u and v .

Building r, e_one and e_two



`r` follows the exact same pattern as `e` from Bob's section:
start empty, loop 3 times, add one random pick each time.
`e_one` is identical to `r` - same code, different name only.
`e_two` isn't a vector, just one number - so no loop at all,
just a single `random.choice([-1, 0, 1])` call.

Building m and m_encoded



v (a single number) is the only place the message can travel,

so m has to be just one bit - not a vector like r or e .

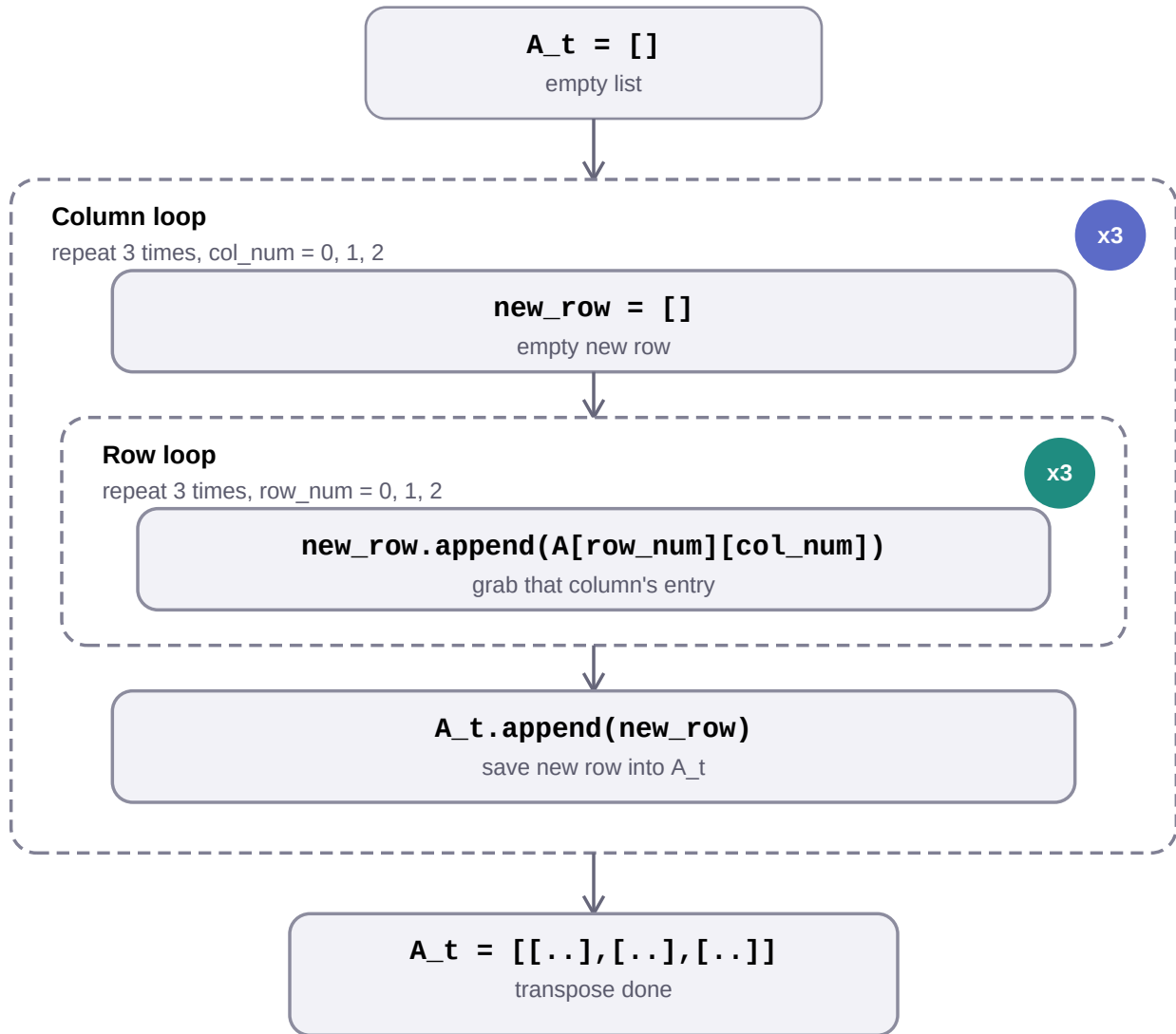
m_{encoded} uses a multiply-as-switch trick: m is 0 or 1, so

$(q // 2) * m$ gives 0 when $m=0$, or the full $q // 2$ when $m=1$.

// means divide then round down (floor division) - e.g.

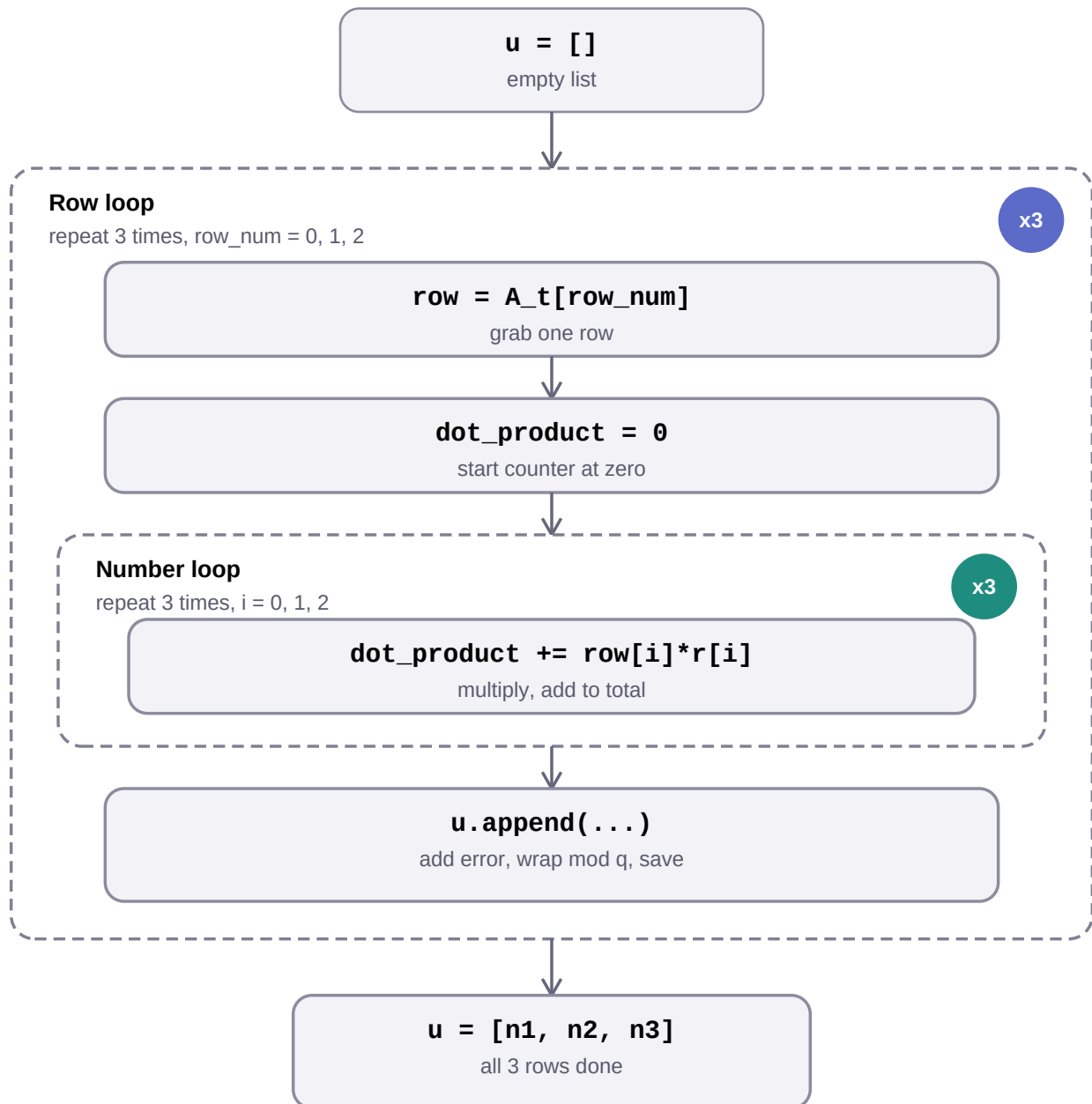
$17 // 2 = 8$, not 8.5.

Building A_t - the transpose of A



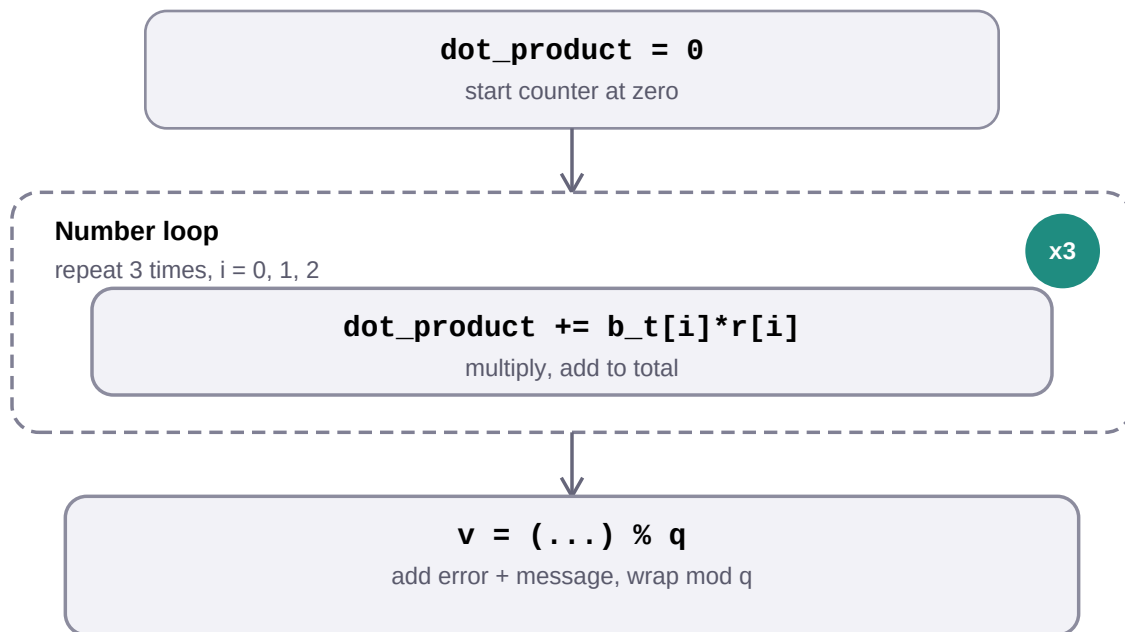
A_t swaps rows and columns. The column loop repeats 3 times (once per new row); for each, the row loop grabs that column's entry from every row of A in turn - A[0][col], A[1][col], A[2][col] - building one new row, which is then saved into A_t. b_t needs none of this: b_t = b, since a vector's transpose doesn't rearrange anything.

Calculating $u = A_t \times r + e_{\text{one}} \pmod q$



Identical pattern to how `b` was built - same 7 lines of code,
just `A` became `A_t`, `S` became `r`, and `e` became `e_one`.
Once you understand `b`, `u` is nothing new, only renamed.

Calculating $v = b_t.r + e_two + m_encoded \pmod q$



v is a single number, so no outer loop is needed - just one dot product (b_t with r), then add e_two , then add $m_encoded$, then wrap mod q .

Ciphertext (u, v) is now ready - Alice sends both to Bob.